Session 3 — Working as a team on GitHub

Git, Python Environments & Coding Best Practices

Guillaume Leveque

Session 3 of 4

Master of Data & AI — MSA-DATB07-03

Recap and the model (10 min)

Recap (hands up)

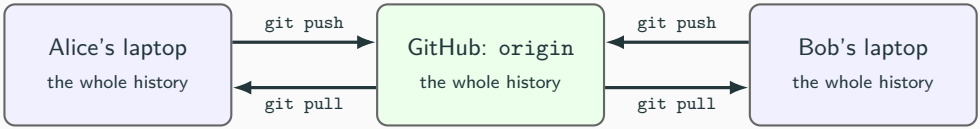
Check yourself — three questions on session 2

1. A merge produced no new commit. What kind of merge was it?
2. You see `<<<<<<< HEAD` in a file. Which three steps finish the merge?
3. Right after a `git clone`, `git branch` shows only `main`, `git branch -a` shows more lines. What are they?

Recap: the answers

1. A **fast-forward**: `main` had not moved since the branch was created, Git only moved the label.
2. Edit the file (keep the right version, delete the three marker lines), `git add` the file, `git commit`.
3. The branches as they are on GitHub: `remotes/origin/...`. Today explains them.

Three repositories, not one



- Every copy holds the whole history. The team agrees to share through one of them, on GitHub: Git calls it `origin`.
- **Nothing moves by itself.** `git commit` stays on your laptop; `git push` sends your commits to GitHub; `git pull` brings back the commits of the others.

Git or GitHub?

Git: a program on your computer

- commits, branches, merges
- `push`, `pull`: talk to a server
- works offline

GitHub: a website

- stores a copy of the repository online
- who may write (collaborators)
- *pull requests*, reviews, Issues

Other websites do the same: GitLab (where a pull request is called a *merge request*), Bitbucket. Learn the workflow once, it works everywhere.

Share: push, clone, pull (25 min)

Step 1 — Put an existing project on GitHub

On GitHub: *New repository*, name `demo-week`, **nothing ticked** (no README, no .gitignore): the project already has its files. GitHub shows the address. Then, in the project folder:

```
cd demo_week
git remote add origin https://github.com/alice/demo-week.git   # give the address a name:
    origin
git remote -v                                                   # check
git push -u origin main          # send main; -u = remember that main goes to origin
```

```
To https://github.com/alice/demo-week.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

After `-u`, a plain `git push` or `git pull` on `main` knows where to go. Reload the GitHub page: the files and the history are there.

origin and main: two different things

name	what it is	think of it as
`origin`	a **repository**: the copy on GitHub	*where*: a short name for its address
`main`	a **branch**: a line of work	*what*: each copy has its own `main`
`origin/main`	`main` as it is on GitHub, last time you looked	shown by `git lg` next to your own `main`

`git push origin main` = "send my branch `main` to the repository `origin`". Two words, two questions: *where* to, and *which* branch.

- `origin` is only a nickname: `git remote add github <address>` would work too. Everybody uses `origin`, and `git clone` chooses it by itself, so every command you read online works as written.
- Why not "main"? Because `main` already names the branch: GitHub has a `main` branch, your laptop has one too.

Step 2 — A teammate clones it

Bob (here: a second folder next to the first one) gets his own copy:

```
cd ..                                                          # next to demo_week
git clone https://github.com/alice/demo-week.git demo_bob      # last word = the folder name
cd demo_bob
git log --oneline -n 2
```

```
Cloning into 'demo_bob'...
1306809 (HEAD -> main, origin/main, origin/HEAD) Merge branch 'feature/rename'
c74c55a Monday is a meeting
```

- `git clone` = create the folder, copy the whole history, name the address `origin`.
- `origin/main` = `main` as it was on GitHub at the last download. It moves only when you `pull` (or `fetch`, appendix).

Step 3 — Bob's terminal: he pushes first

Two terminals side by side in VS Code: **Alice** on the left (folder `demo_week`), **Bob** on the right (folder `demo_bob`). Here, Bob's terminal:

```
pwd                                    # .../demo_bob
# notes.txt: add the line "Friday: demo" (VS Code), save
git add notes.txt
git commit -m "Add Friday"
git push                               # main goes to origin (thanks to -u at the clone)
```

```
To https://github.com/alice/demo-week.git
   1306809..ce30d5e  main -> main
```

`1306809..ce30d5e`: `main` on GitHub moved from Alice's last commit to Bob's new one. Alice does not have it yet.

Step 3 — Alice's terminal: her push is refused

Meanwhile, without pulling, Alice worked too. Alice's terminal:

```
pwd                                          # .../demo_week
# todo.txt: add the line "buy eggs" (VS Code), save
git add todo.txt
git commit -m "Add eggs"
git push
```

```
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/alice/demo-week.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. ... If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
```

GitHub has Bob's commit, Alice does not. Accepting her push would *replace* Bob's work: GitHub refuses. Git even says what to do: `git pull`.

Step 4 — Alice's terminal: pull, then push

```
git pull          # download Bob's commit, MERGE it into my main (VS Code: save, close)
git push          # now accepted
git lg
```

```
*   ca8fba2 (HEAD -> main, origin/main) Merge branch 'main' of https://github.com/alice/demo-week
|\
| * ce30d5e Add Friday
* | ef5ea27 Add eggs
|/
*   1306809 Merge branch 'feature/rename'
```

`git pull` = download + merge: the three-way merge of session 2, with Bob's line of work. Same lines changed by both? A conflict, solved as in session 2.

Pitfall — never `git push --force`

`--force` makes GitHub accept your version and **deletes Bob's commits**. On a shared branch: never.

Check yourself (hands up)

Bob pushed two commits on `main`. On Alice's laptop, `main` and `origin/main` are on the same old commit, and she has no new commit.

Check yourself — what happens?

1. Alice runs `git pull`: what changes in her files? Fast-forward or three-way merge?
2. Instead, Alice first makes a commit, then runs `git push`: what does GitHub answer?
3. In case 2, which two commands solve it, in which order?

Check yourself: the answers

1. Her files get Bob's two commits. A **fast-forward**: she had nothing new, her `main` just moves forward to Bob's last commit.
2. **Rejected** ("fetch first"): GitHub has commits she does not have.
3. `git pull` (three-way merge, maybe a conflict), then `git push`.

The pull request (30 min)

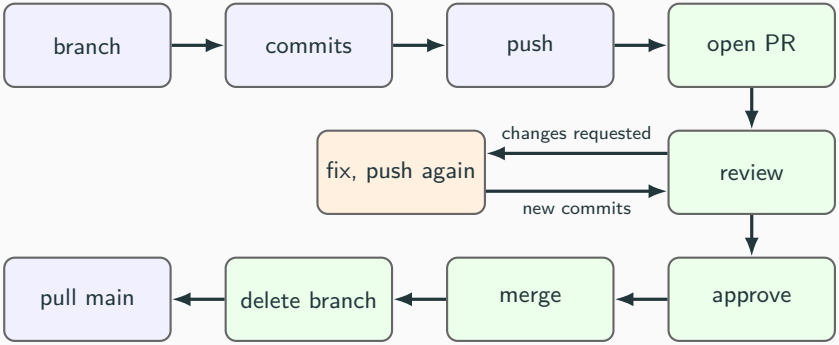
Why not push on main directly?

- A change nobody read is a change nobody understands next month.
- Half-finished work on `main` breaks the project for everybody.
- A second pair of eyes catches bugs, and *two* people now know the code.

Pull request (PR)

"Please merge my branch into `main`." A GitHub page attached to a branch: the changes, a discussion, the reviews, and a *Merge* button.

The cycle of a pull request



Blue: on your laptop. Green: on GitHub. Orange: the loop that makes the code better.

Step 5 — The author: a branch, then push it

```
git switch main
git pull                                  # start from the latest main
git switch -c feat/saturday               # one task = one branch
# edit notes.txt in VS Code: add "Saturday: rest", save
git add notes.txt
git commit -m "feat: add Saturday"
git push -u origin feat/saturday          # the branch goes to GitHub
```

```
remote: Create a pull request for 'feat/saturday' on GitHub by visiting:
remote:      https://github.com/alice/demo-week/pull/new/feat/saturday
To https://github.com/alice/demo-week.git
 * [new branch]      feat/saturday -> feat/saturday
```

Open the link (or the yellow *Compare & pull request* button on GitHub).

Step 6 — Open the pull request

- **base**: `main` ← **compare**: `feat/saturday`
- **Title**: what the PR does
- **Description**: what, and why
- **Reviewers** (right): a teammate
- *Create pull request*

```
## What
Adds Saturday to the notes.

## Why
The plan covers the whole week.
Closes #3
```

`Closes #3`: when the PR is merged, Issue number 3 is closed automatically (Issues: two slides further).

Step 7 — The reviewer: what to look at

Tab *Files changed*: green lines added, red lines removed. Ask yourself:

1. **Does it do what the description says?** Read the Issue and the *What / Why*.
2. **Is it right?** Edge cases: an empty list, a missing value, a zero.
3. **Is it tested?** A new function comes with a test that checks something useful.
4. **Is it clear?** Names that say what things are; no copy-paste of existing code.
5. **Is everything there?** README updated, nothing left over (`print` for debugging, a file that should not be committed).

Not sure? Run it: `git switch` to the branch (after `git pull`), then `python -m pytest`.

Step 7 — The reviewer: write useful comments

Click on a line in *Files changed*, write, *Add single comment* (or *Start a review*).

a question	"What happens if `rows` is empty? Division by zero?"
a bug	"Line 12 adds the quantity, not quantity × price."
a suggestion	"`TASKS.md` calls it `average_order_value`: rename it?"
a missing piece	"The README does not list the new function yet."
a precise thanks	"Good idea, the test for the empty list."

Then *Review changes*: **Approve** (ready), or **Request changes** (something must change before the merge: a bug, a missing test).

Pitfall — "Looks good" is not a review

A review without a comment checked nothing. Talk about the code, never the person: "this line", "we could", a question rather than "you did it wrong".

Step 7 — After the approval: merge

The author

- changes requested? edit, commit, `git push` on the *same* branch: the pull request updates itself, the reviewer looks again
- approved: *Merge pull request* → *Confirm merge*
- *Delete branch*: the branch is merged, its label is not needed any more

Then everybody

- `git switch main`
- `git pull`

Everybody's `main` now contains the merged work, and the Issue is closed (`Closes #3`).

Protect main

Settings → *Branches* → *Add classic branch protection rule*, pattern `main`:

- **Require a pull request before merging**: a direct `git push` on `main` is refused.
- **Require approvals**: 1 approval at least.
- **Do not allow bypassing the above settings**: the rule applies to the owner too.

```
remote: error: GH006: Protected branch update failed for refs/heads/main.
remote: - Changes must be made through a pull request.
```

The team rule "nobody pushes on `main`" is now enforced by GitHub itself. Your team repository gets it in TD3.

Issues: who does what

- An **Issue** is a card in the to-do list of the repository: a task, a bug, a question. GitHub numbers it: #3.
- *Assignees*: the person who does it. The whole team sees who does what.
- `Closes #3` in a pull request: the Issue closes when the PR is merged.

In TD3: one Issue per task, each member assigns themselves, each pull request closes its Issue.

A conflict in a pull request (15 min)

Step 8 — Two pull requests change the same line

```
git switch main && git pull
git switch -c fix/monday-morning        # line 1 -> "Monday: meeting at 9", commit, push, PR
    #2
git switch main
git switch -c fix/monday-room           # line 1 -> "Monday: meeting in room B", commit,
    push, PR #3
```

- PR #2 is approved and merged first: `main` now has "meeting at 9".
- PR #3 now shows: *This branch has conflicts that must be resolved*. The *Merge* button is grey.
- Both changed line 1 of `notes.txt`, in two different ways: the conflict of session 2, now between two teammates.

Step 9 — Resolve it on your branch

The author of PR #3 brings the new `main` into the branch, and resolves there:

```
git switch main
git pull                          # main with PR #2
git switch fix/monday-room
git merge main                    # bring main INTO my branch
```

```
CONFLICT (content): Merge conflict in notes.txt
```

```
# VS Code: line 1 = "Monday: meeting at 9 in room B", delete the 3 marker lines, save
git add notes.txt
git commit                        # the merge commit: save, close the tab
git push                          # PR #3 updates itself: no conflict any more, merge it
```

Also possible: the *Resolve conflicts* button on the pull request page, which creates the same merge commit.

Step 10 — Read the team history

```
git switch main && git pull && git lg
```

```
*   5e1c9a0 (HEAD -> main, origin/main) Merge pull request #3 from alice/fix/monday-room
|\
| *   b7d2e41 Merge branch 'main' into fix/monday-room
| |\
| |/
|/|
* |   9a4f310 Merge pull request #2 from alice/fix/monday-morning
|\ \
| * | 3c8e2d7 fix: Monday meeting at 9
|/ /
| * 61f0b9e fix: Monday meeting in room B
|/
*   4d2a8c1 Merge pull request #1 from alice/feat/saturday
```

- `Merge pull request #n`: one per merged pull request (the *Merge* button).
- `Merge branch 'main' into ...`: where a conflict was resolved, on the branch, before its pull request.

Fewer conflicts, smaller conflicts

- **Pull before you start**, every time: `git switch main`, `git pull`.
- **Small pull requests, merged often.** A branch that lives two days conflicts less than one that lives two weeks.
- **Talk.** Two people changing the same function should know it before, not after.
- A conflict is not a mistake: it is Git asking a question. Answer it together if you are not sure.

Commit messages (5 min)

A readable history: Conventional Commits

```
<type>(<optional scope>): <what the commit does, short>

feat(metrics): add average_order_value
fix(loader): read quantity as a number
docs: add a data dictionary to the README
```

`feat`	a new feature
`fix`	a bug fix
`docs`	documentation only
`test`	tests only
`chore`	configuration, dependencies

`git log --oneline` becomes readable at a glance. Rewrite: `fixed stuff`, `Update analysis.py`, `WIP`.

Wrap-up (5 min)

Team rules

- `git switch main` and `git pull` before you start a task
- never push on `main`: the protection is on
- one task = one Issue = one branch = one pull request
- ask a teammate for a review; review the others quickly, with real comments
- small pull requests, merged often: fewer conflicts
- clear messages: `feat:`, `fix:`, `docs:`
- a conflict? `git merge main` on your branch, resolve, push; ask the author of the other change if you are not sure

Safe or not: the summary, with GitHub

command	effect	risk
`git clone`, `git remote -v`	copy a repository, show its address	**safe**
`git push`	add my commits on GitHub (refused if I am behind)	**safe**
`git pull`	download and merge the commits of the others	**safe** (may conflict)
`git merge main` on my branch	bring the new `main` into my branch	**safe** (may conflict)
`git reset --hard origin/main`	drop my local commits, back to GitHub's `main`	**destroys work** (only for commits never pushed)
`git push --force`	replace the branch on GitHub by mine	**never on shared history**

Read this repository

On GitHub, `main` of the team, on Monday and on Tuesday:

```
Monday                                    Tuesday
c3d4e5f fix(loader): read quantity        4b2c9d1 feat(metrics): add top_products
9e8f7a6 feat(metrics): add ...            9e8f7a6 feat(metrics): add ...
fb53690 docs: add contributing guide      fb53690 docs: add contributing guide
```

Check yourself — the kind of question of the quiz

1. Bob's commit `c3d4e5f` disappeared from GitHub. Alice's normal push was refused on Monday evening: why? Which command did she use to push anyway?
2. What should she have done instead?
3. Which protection setting would have refused her push?

Read this repository: the answers

1. Bob had pushed `c3d4e5f`; Alice did not have it, so GitHub refused her push ("fetch first"). She then used `git push --force`: GitHub's `main` was replaced by hers, without Bob's commit.
2. `git pull` (get Bob's commit and merge it, resolve the conflict if there is one), then `git push`.
3. Protect `main`: *Require a pull request before merging* refuses every direct push on `main`, forced or not.

Bob's commit is not lost forever: it is still on Bob's laptop, and he can push it again.

Take-aways and TD3

Remember

- three copies: mine, `origin`, theirs; nothing moves by itself
- `push` sends, `pull` brings back and merges
- push refused: `pull`, then `push`; never `--force`
- branch → push → PR → review → merge → pull
- conflict in a PR: `git merge main` on your branch, resolve, push

TD3 (1 h 30), in teams of 3 or 4:

1. your team and its repository (20 min)
2. split the work with Issues (10 min)
3. one task each, through pull requests; two planned conflicts (45 min)
4. check the protection (10 min)
5. look at your work on GitHub (5 min)

Next session: Python environments (venv, pip, conda, uv), in the same team repository.

Appendix: for the curious

Appendix: for the curious

Not needed for the quiz

The next slides are extras: they are not presented in the lecture, and the quiz does not ask about them.

- SSH keys: connect to GitHub without a password or a token
- `git fetch`: look before you merge
- the three merge buttons of GitHub
- forks: contribute to a repository you cannot push to

Appendix — SSH keys

```
ssh-keygen -t ed25519 -C "alice.martin@student.example.edu"   # Enter: default file
cat ~/.ssh/id_ed25519.pub        # the PUBLIC key: one line starting with ssh-ed25519
ssh -T git@github.com            # test: "Hi alice! You've successfully authenticated"
```

- Give the `.pub` line to GitHub: *Settings* → *SSH and GPG keys* → *New SSH key*. The private key (`id_ed25519`, without `.pub`) never leaves your laptop.
- Then use the SSH addresses: `git@github.com:alice/demo-week.git`.
- Network blocks SSH (port 22)? GitHub also answers on port 443:

```
printf 'Host github.com\n  Hostname ssh.github.com\n  Port 443\n  User git\n' >> ~/.ssh/
    config
```

Appendix — git fetch: look before you merge

```
git fetch                              # download, change NOTHING of yours: origin/main
    moves
git log --oneline main..origin/main    # the commits GitHub has and I do not
git merge origin/main                  # then merge them, if you want
git fetch --prune                      # forget the origin/... branches deleted on GitHub
```

`git pull` = `git fetch` + `git merge`. `fetch` alone never surprises you with a merge or a conflict.

Appendix — The three merge buttons

button	what lands on `main`	when
Create a merge commit	the commits of the branch + a merge commit	the default, used in this course
Squash and merge	one single commit with all the changes	a branch with many small commits
Rebase and merge	the commits copied, no merge commit	teams that want a straight history

A team decision, written in `CONTRIBUTING.md`.

Appendix — Forks

Branch when you may write in the repository (your team). **Fork** when you may not (an open-source library, another team): GitHub's *Fork* button makes a full copy *under your account*, where you can push.

```
git clone https://github.com/alice/some-library.git     # clone YOUR fork
git remote add upstream https://github.com/org/some-library.git   # the original
git switch -c fix/typo
# ... edit, commit ...
git push -u origin fix/typo      # to your fork; then a PR from your fork to the original
git fetch upstream               # later: get the news of the original
git merge upstream/main
```